UCT403: SOFTWARE DESIGN WITH UML

L	T	P	Cr
2	0	2	3.0

Course Objectives: To apply principles of software development and evolution. To specify, abstract, verify, validate, plan, develop and manage large software and learn emerging trends in software engineering

Introduction to an Object Oriented Technologies and the UML Method. : Software development process: The Waterfall Model vs. The Spiral Model. The Software Crisis, description of the real world using the Objects Model. Classes, inheritance and multiple configurations. Quality software characteristics. Description of the Object Oriented Analysis process vs. the Structure Analysis Model.

Introduction to the UML Language: Standards. Elements of the language. General description of various models. The process of Object Oriented software development. Description of Design Patterns. Technological Description of Distributed Systems.

Requirements Analysis Using Case Modeling: Analysis of system requirements. Actor definitions. Writing a case goal. Use Case Diagrams. Use Case Relationships.

Transfer from Analysis to Design in the Characterization Stage: Interaction Diagrams. Description of goal. Defining UML Method, Operation, Object Interface, Class. Sequence Diagram. Finding objects from Flow of Events. Describing the process of finding objects using a Sequence Diagram. Describing the process of finding objects using a Collaboration Diagram.

The Logical View Design Stage: The Static Structure Diagrams. : The Class Diagram Model. Attributes descriptions. Operations descriptions. Connections descriptions in the Static Model. Association, Generalization, Aggregation, Dependency, Interfacing, Multiplicity.

Package Diagram Model: Description of the model. White box, black box. Connections between packagers. Interfaces. Create Package Diagram. Drill Down.

Dynamic Model: State Diagram / Activity Diagram: Description of the State Diagram. Events Handling. Description of the Activity Diagram. Exercise in State Machines.

Component Diagram Model & Deployment Model: Physical Aspect. Logical Aspect. Connections and Dependencies. User face. Initial DB design in a UML environment. Processors. Connections. Components. Tasks. Threads. Signals and Events.

Laboratory work: Implementation of Software Engineering concepts and exposure to CASE tools like Rational Software suit, Turbo Analyst, Silk Suite. Follow entire SDLC depending on project domain.

Course learning outcomes (CLOs):

Upon the completion of the course, the student will be able to:

1. Comprehend software development all modern process models, including traditional models like waterfall and spiral.
2. Demonstrate the use of software life cycle through requirements gathering, choice of process model and design model.
3. Apply and use various UML models for software analysis, design and testing.
4. Acquire knowledge about the concepts of application of case tools and configuration management for software development.

Text Books:

1. Object-Oriented Software Engineering: using UML, Patterns, and Java. Bernd Bruegge and Allen H. Dutoit.
2. Pressman S. R. and Maxim R. B., Software Engineering, A Practitioner‟s Approach, McGraw Hill International (2015) 8th Edition.
3. Sommerville I., Software Engineering, Addison-Wesley Publishing Company (2011) 9th Edition

Reference Books:

1. Foster C. E., Software Engineering: A Methodical Approach, Apress (2014)1st ed.
2. Booch G., Rambaugh J., Jacobson I., The Unified Modeling Language User Guide (2005) 2nd Edition.